Лабораторная работа №6: Реализация Factory Method и Builder для динамического конфигурирования бизнес-объектов и отчетов

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Неделя: 6 из 10
Формат: Лабораторная работа, 3 академических часа (135 минут)
Среда выполнения: Python 3.12+, pytest, Git, VS Code / PyCharm

---

1. Цель и контекст работы

Цель работы: Освоить на практике применение порождающих паттернов проектирования GoF (**Factory Method** и **Builder**) для устранения жесткого зацепления при создании объектов и провести рефакторинг антипаттерна **Singleton**.
В системе городского транзита Avtobys: 1. Выпуск билетов зависит от устройства: на валидаторе генерируется динамический QR-код, на турникете считывается криптографический токен NFC, а контроллер на линии проверяет разовый бумажный фискальный чек. Создание билетов должно быть инкапсулировано через **Factory Method**. 2. В конце рейса формируется сводный отчет диспетчера: маршрут, водитель, временные интервалы, разбивка выручки по платежным провайдерам, журнал сбоев и аномалий. Конструирование такого комплексного объекта требует паттерна **Builder**. 3. Устаревший глобальный синглтон конфигурации валидатора GlobalConfig должен быть демонтирован в пользу внедрения зависимостей.

---

2. Структура репозитория и требования к сдаче

Работа выполняется в папке lab06_creational_patterns/:

```
lab06_creational_patterns/
├── factory/
│   ├── __init__.py
│   ├── tickets.py      # Иерархия билетов (QR, NFC, Paper)
│   └── issuers.py      # Фабричные методы (TicketIssuers)
├── builder/
│   ├── __init__.py
│   ├── model.py        # Иммутабельная модель DailyShiftReport
│   └── builder.py      # Реализация ShiftReportBuilder с валидацией
├── legacy_fix/
│   ├── __init__.py
│   └── config.py       # Рефакторинг Singleton -> DI-ready Config
├── tests/
│   ├── __init__.py
│   ├── test_factory.py # Тесты фабрик билетов
```

```
│   └── test_builder.py # Тесты пошаговой сборки отчета
└── README.md           # Анализ преимуществ примененных паттернов
```

Критерии оценки (20 баллов):

Блок	Задание	Баллы
Блок 1	Реализация Factory Method для полиморфной эмиссии билетов	5
Блок 2	Разработка ShiftReportBuilder с текучим интерфейсом (Fluent API)	7
Блок 3	Рефакторинг глобального синглтона конфигурации	4
Блок 4	Тестирование юнит-тестами pytest с проверкой инвариантов билдера	4

---

3. Практические задания

Блок 1. Реализация Factory Method (5 баллов)

В папке factory/: 1. Определите базовый контракт билета Ticket с абстрактными методами: - get_token() -> str - calculate_vat() -> Decimal (расчет НДС 12%) 2. Реализуйте 3 типа билетов: - QrTicket: содержит qr_payload и время жизни 15 минут. - NfcTicket: содержит серийный номер чипа и криптографическую подпись. - PaperSlipTicket: содержит фискальный чек и номер терминала печати. 3. Создайте абстрактный эмитент TicketIssuer с фабричным методом create_ticket(fare: Decimal) -> Ticket и конкретные фабрики QrTicketIssuer, NfcTicketIssuer, PaperTicketIssuer. 4. Клиентский метод issue_ticket_to_passenger(issuer: TicketIssuer, fare: Decimal) должен работать только с абстрактным эмитентом!

Блок 2. Разработка ShiftReportBuilder (7 баллов)

В папке builder/: Спроектируйте пошаговый строитель финансового отчета рейса:

```
class ShiftReportBuilder:
    def set_shift_info(self, route_id: str, driver_id: str, shift_date: date) -> "ShiftReportBuilder": ...
    def add_qr_revenue(self, amount: Decimal, count: int) -> "ShiftReportBuilder": ...
    def add_card_revenue(self, amount: Decimal, count: int) -> "ShiftReportBuilder": ...
    def add_cash_revenue(self, amount: Decimal, count: int) -> "ShiftReportBuilder": ...
    def record_incident(self, time_str: str, description: str) -> "ShiftReportBuilder": ...
    def build(self) -> DailyShiftReport: ...
```

Обязательная валидация в методе build(): - route_id и driver_id обязательны (выбрасывает IncompleteReportError, если не заданы). - Суммарная выручка должна быть ≥ 0.00. - Результирующий объект DailyShiftReport должен быть неизменяемым (frozen=True).

Блок 3. Рефакторинг Singleton (4 балла)

Вам дан легаси-класс GlobalConfig, реализованный через классический синглтон:

```
# АНТИПАТТЕРН: глобальный мутабельный синглтон
class GlobalConfig:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.gateway_url = "https://api.avtobys.kz"
        return cls._instance
```

Задание: 1. Перепишите класс в обычный неизменяемый объект конфигурации AppConfig с загрузкой из словаря или переменных окружения. 2. Продемонстрируйте, как этот конфиг передается в сервисы через Constructor Injection, устраняя скрытые глобальные зависимости.

Блок 4. Автоматическое тестирование (pytest) (4 балла)

1. Проверьте полиморфизм фабричного метода: каждый эмитент создает именно свой тип билета.
2. Проверьте билдер: сборка отчета с валидными данными завершается успешно, а пропуск обязательного поля set_shift_info приводит к исключению IncompleteReportError.
3. Напишите тест параллельной работы: создание двух независимых отчетов через разные билдеры не приводит к смешиванию данных.

```
pytest tests/ -v
```

---

4. Контрольные вопросы для защиты работы

1. Чем фабричный метод (Factory Method) принципиально отличается от простой статической фабричной функции?
2. В каких случаях использование Builder является избыточным усложнением кода?
3. Почему при параллельном запуске тестов (например, через pytest -n 4) паттерн Singleton приводит к случайным падениям тестов?
4. Как фабричный метод поддерживает принцип Open/Closed (OCP)?